

KON-TIKI FINAL TECHNICAL REPORT: COMPASS

About the project

the goal of the project is to give us an accurate reading of the rocket's state eg velocity, position, orientation, and acceleration. this is so that when the time is right we can deploy the parachute and land the rocket safely without tearing it apart. it will also give us a better understanding of the rocket and how it behaves in the air. and more easily recover the rocket. We do this by using a combination of sensors and a microcontroller to process the data and send it to the ground station as well as store it on the rocket for redundancy and flight data analysis.

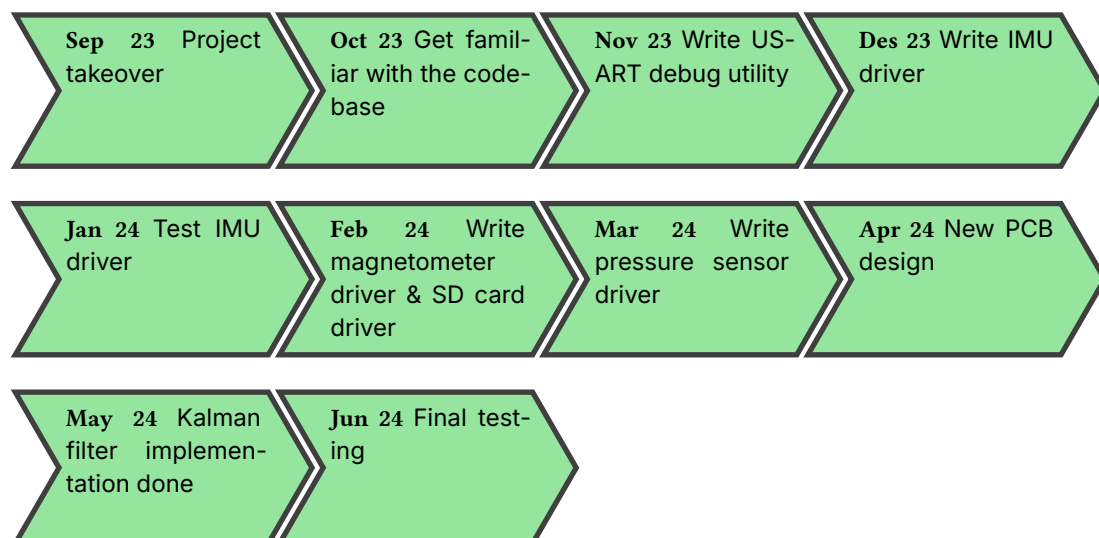
Summary

The project is a success, we have a working prototype that has been tested in the field and in the lab. the major takeaway is that the system works well for altitude estimation and orientation. but x and y velocity and position is prone to drift. The reason for this is the lack of absolute positioning and velocity sensors like GPS and airspeed sensors. The orientation and altitude states have such sensors and are therefore more accurate. Despite this, the system does a good job of giving a rough estimate of the velocity and position. With so called "dead reckoning" we can estimate the position and velocity without the need of telecommunication that GPS needs.

Development

As the project was based on previous projects the specs were already set in stone. and they did not change much during the development of the project. however the last iteration swapped the magnetometer for a more accurate one. but the overall design and specs stayed the same.

Timeline



Modeling

The navigation system is based on a state-space representation of the system. The state of the system is defined as the orientation, velocity, and altitude of the rocket. The dynamics of the system are described by a set of differential equations. The sensors are used to measure the state of the system, and the Kalman filter is used to estimate the state of the system. To estimate the state of the rocket, we must first build a model of the rocket's dynamics and the measurement models of the sensors. The dynamics of the rocket can be described by a set of differential equations. The state of the rocket is defined as a vector of variables that describe the orientation, velocity, and altitude of the rocket.

The general form of a dynamical system can be stated as

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{u}) + \mathbf{w}_d \\ \hat{\mathbf{y}} &= \mathbf{h}(\mathbf{x}, \mathbf{u}) + \mathbf{w}_n\end{aligned}$$

where $\mathbf{x}$ is the state vector, $\mathbf{u}$ is the control vector, $\mathbf{w}_d$ is the system disturbance, $\mathbf{y}$ is the measurement vector, $\mathbf{w}_n$ is the measurement noise, $\mathbf{f}$ is the dynamics of the system. For linear systems we can use linear algebra to express our system

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{F}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}_d \\ \hat{\mathbf{y}} &= \mathbf{H}\mathbf{x} + \mathbf{D}\mathbf{u} + \mathbf{w}_n\end{aligned}$$

we can also discreteize our system (which is what we must do in order to use our computer algorithms)

$$\begin{aligned}\mathbf{x}_{t+1} &= \mathbf{f}_d(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_d \\ \hat{\mathbf{y}}_t &= \mathbf{h}_d(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_n\end{aligned}$$

(1)

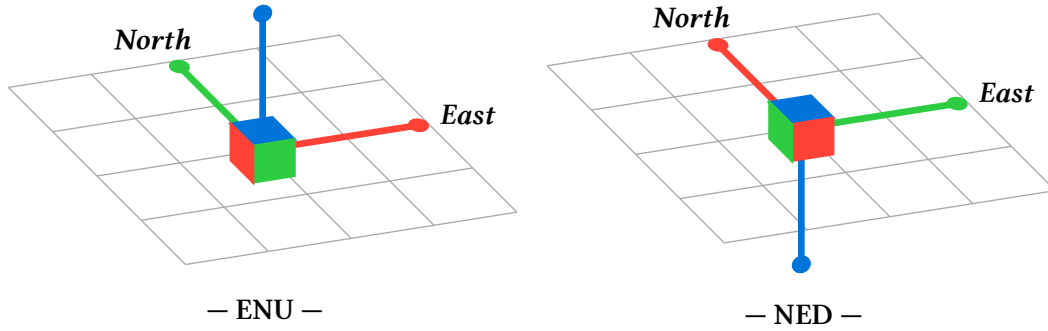
$$\begin{aligned}\text{in the linear case} \\ \mathbf{x}_{t+1} &= \mathbf{F}_d\mathbf{x}_t + \mathbf{B}_d\mathbf{u}_t + \mathbf{w}_d \\ \hat{\mathbf{y}}_t &= \mathbf{H}_d\mathbf{x}_t + \mathbf{D}_d\mathbf{u}_t + \mathbf{w}_n\end{aligned}$$

in our case the vector $\mathbf{u}$ is 0 since we have no controll over the rocket as far as the navigation system is concerned. The state vector $\mathbf{x}$ is defined as

$$\mathbf{x} = \begin{pmatrix} \text{orientation} \in \mathbb{R}^4 \\ \text{velocity} \in \mathbb{R}^3 \\ \text{altitude} \in \mathbb{R} \end{pmatrix}$$

Coordinate systems

Before we can define the state of the rocket, we must agree upon the coordinate systems. There are a some popular ones such as the NED (North-East-Down) and ENU (East-North-Up). These coordinate systems define the choice of basis vectors (the directions of the axes).



We use the ENU coordinate system where the x-axis points east, the y-axis points north, and the z-axis points up.

Orientation and quaternions

The orientation of the rocket is represented by a quaternion. A quaternion can be thought of as an extension of complex numbers. It can be used to represent rotations and orientations in 3D space, depending on the context. The quaternion is defined as

$$\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$$

where a, b, c, d are real numbers and $\mathbf{i}, \mathbf{j}, \mathbf{k}$ are the imaginary units. It can be efficiently represented on a computer as a 4D vector.

$$\mathbf{q} = \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}$$

Without diving too deep into the math, the quaternion can be used to represent a rotation

$$\hat{\mathbf{v}} = \mathbf{q}\mathbf{v}\mathbf{q}^{-1}$$

$\hat{\mathbf{v}}$ is the result of rotating $\mathbf{v}$ by $\mathbf{q}$. where $\mathbf{v}$ and $\hat{\mathbf{v}}$ is 3D vectors expressed as a quaternion with the scalar part being 0.

$$\mathbf{v} = \begin{pmatrix} 0 \\ v_1 \\ v_2 \\ v_3 \end{pmatrix} \quad \hat{\mathbf{v}} = \begin{pmatrix} 0 \\ \hat{v}_1 \\ \hat{v}_2 \\ \hat{v}_3 \end{pmatrix}$$

In this example we have used the complex conjugate and the quaternion multiplication operations.

Quaternion conjugate

For a unit quaternion the inverse is equal to the conjugate $\mathbf{q}^{-1} = \bar{\mathbf{q}}$

$$\bar{\mathbf{q}} = \begin{pmatrix} a \\ -b \\ -c \\ -d \end{pmatrix}$$

Quaterinion multiplication

$$\mathbf{q}_1 \mathbf{q}_2 = \begin{pmatrix} a_1 a_2 - b_1 b_2 - c_1 c_2 - d_1 d_2 \\ a_1 b_2 + b_1 a_2 + c_1 d_2 - d_1 c_2 \\ a_1 c_2 - b_1 d_2 + c_1 a_2 + d_1 b_2 \\ a_1 d_2 + b_1 c_2 - c_1 b_2 + d_1 a_2 \end{pmatrix}$$

This operation is in general not commutative, so the order of the quaternions matter $\mathbf{q}_1 \mathbf{q}_2 \neq \mathbf{q}_2 \mathbf{q}_1$. But it is associative meaning that $(\mathbf{q}_1 \mathbf{q}_2) \mathbf{q}_3 = \mathbf{q}_1 (\mathbf{q}_2 \mathbf{q}_3)$.

Quaterinion normalization

The magnitude of a quaternion is defined as

$$\|\mathbf{q}\| = \sqrt{\mathbf{q}\bar{\mathbf{q}}} = \sqrt{\bar{\mathbf{q}}\mathbf{q}} = \sqrt{a^2 + b^2 + c^2 + d^2}$$

we want to *normalize* the quaternion so that it has a magnitude of 1, and consequently

$$\mathbf{q}\bar{\mathbf{q}} = \|\mathbf{q}\|^2 = 1$$

we can normalize the quaternion by dividing it by its magnitude

$$\mathbf{q}_u = \frac{\mathbf{q}}{\|\mathbf{q}\|}$$

Quaterinion kinematics

The derivative of a quaternion is defined as

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \boldsymbol{\omega}$$

where $\boldsymbol{\omega}$ is the angular velocity of our system. By using a gyro sensor, we can obtain $\boldsymbol{\omega} = \begin{pmatrix} 0 \\ \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}$. With this equation, we can integrate the angular velocity to get the orientation of the rocket.

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \frac{1}{2} \mathbf{q} \boldsymbol{\omega} dt$$

Velocity and altitude

The velocity of the rocket is represented as a 3D vector

$$\mathbf{v} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}$$

the velocity is obtained by integrating the acceleration of the rocket. However, we want the velocity in respect to the world frame, and thus we cannot directly integrate each component of the acceleration in a decoupled manner. What if the rocket is tilted say along the x-axis? In this case, the acceleration along the z-axis will also have a component along the x-axis in the world frame. The rocket frame is rotated with respect to the world frame. To combat this, we must find a mapping or a translation if you will, between the rocket frame and the world frame. Lucky for us, we have the quaternion that can be used to rotate the acceleration vector to the world frame. So the velocity is obtained by integrating the acceleration in the world frame.

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{q}\mathbf{a}\bar{\mathbf{q}} dt$$

Finally, the altitude is a scalar denoting the height of the rocket above the sea level in meters. We can simply integrate the velocity along the z-axis to get the altitude.

$$z_{t+1} = z_t + v_z dt$$

Dynamics

As we have seen, the dynamics is governed by the following equations

$$\mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{pmatrix} \mathbf{q}_{t+1} = \mathbf{q}_t + \frac{1}{2}\mathbf{q}\boldsymbol{\omega} dt \\ \mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{q}\mathbf{a}\bar{\mathbf{q}} dt \\ z_{t+1} = z_t + v_z dt \end{pmatrix}$$

we can think of the input $\mathbf{u}$ as being our $\boldsymbol{\omega}$ and $\mathbf{a}$ from the sensors.

Orientation dynamics

Lets first consider the orientation part. This equation is in fact a linear equation in the state variable $\mathbf{q}$, However, We want to express the equation in a matrix-vector multiplication form

$$\mathbf{q}_{t+1} = \mathbf{F}_q \mathbf{q}_t$$

Where $\mathbf{F}_q$ is a matrix. We can achieve this by expanding our quaternion kinematics equation to get

$$\begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{pmatrix} + \begin{pmatrix} q_1\omega_0 - q_2\omega_x - q_3\omega_y - q_4\omega_z \\ q_1\omega_x + q_2\omega_0 + q_3\omega_z - q_4\omega_y \\ q_1\omega_y - q_2\omega_z + q_3\omega_0 + q_4\omega_x \\ q_1\omega_z + q_2\omega_y - q_3\omega_x + q_4\omega_0 \end{pmatrix} \frac{1}{2} dt$$

and we can see that this can be expressed as a matrix-vector multiplication $\mathbf{F}_q \mathbf{q}$ where the matrix $\mathbf{F}_q$ is

$$\mathbf{F}_q = \begin{pmatrix} 1 & -\frac{1}{2}\omega_x dt & -\frac{1}{2}\omega_y dt & -\frac{1}{2}\omega_z dt \\ \frac{1}{2}\omega_x dt & 1 & \frac{1}{2}\omega_z dt & -\frac{1}{2}\omega_y dt \\ \frac{1}{2}\omega_y dt & -\frac{1}{2}\omega_z dt & 1 & \frac{1}{2}\omega_x dt \\ \frac{1}{2}\omega_z dt & \frac{1}{2}\omega_y dt & -\frac{1}{2}\omega_x dt & 1 \end{pmatrix}$$

Velocity dynamics

As with the orientation dynamics we want to express the equations by a matrix-vector product. However the velocity dynamics is not linear since the quaternion components are coupled in a non-linear way. We get expressions like q_1^2 and $q_1 q_2$ which are non-linear. In order to get the matrix we want we can linearize, by taking the first derivative of our velocity equation we get the jacobian matrix

$$\mathbf{F}_{vq} = 2 \begin{pmatrix} q_1 a_x + q_3 a_z - q_4 a_y & q_2 a_x + q_3 a_y + q_4 a_z & q_1 a_z - q_3 a_x + q_2 a_y & q_4 a_x - q_1 a_y + q_2 a_z \\ q_1 a_y + q_4 a_x - q_2 a_z & q_3 a_x - q_2 a_y - q_1 a_z & q_3 a_y + q_2 a_x + q_4 a_z & q_3 a_z - q_4 a_y + q_1 a_x \\ q_1 a_z + q_2 a_y - q_3 a_x & q_4 a_x - q_2 a_z + q_1 a_y & q_4 a_y - q_3 a_z - q_1 a_x & q_4 a_z + q_2 a_x + q_3 a_y \end{pmatrix} dt$$

$$\mathbf{F}_{vv} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Position dynamics

This one is simple, to express the position dynamics in a matrix-vector multiplication form we can simply write

$$\mathbf{F}_a = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & dt & 1 \end{pmatrix}$$

$$\mathbf{z}_{t+1} = \mathbf{F}_a \begin{pmatrix} \vdots \\ v_z \\ \mathbf{z}_t \end{pmatrix}$$

Putting it all together we get the dynamics matrix $\mathbf{F}$

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \mathbf{F} = \begin{pmatrix} \mathbf{F}_q & \mathbf{0} & \mathbf{0} \\ \mathbf{F}_{vq} & \mathbf{F}_{vv} & \mathbf{0} \\ \dots & \mathbf{F}_a & \dots \end{pmatrix}$$

Measurement models

Systems have different degrees of observability. Some states are observable from the sensors, while others are not. The orientation of the rocket can be measured by sensors such as magnetometers, sun trackers and star trackers. The altitude can be directly measured by barometers, and both altitude and velocity can be measured by GPS. Though GPS is not used in this system. With the sensors we have, we still need a way to translate between the sensor readings and our state model. That is what measurement models are for. They describe how the state of the system is related to the sensor readings.

Orientation measurement

To measure the orientation of the rocket, we use a magnetometer. The magnetometer measures the magnetic field of the earth. The magnetic field of the earth is known and is going nowhere (anytime soon), so it can be used as a reference. We need a reading of the magnetic field when the rocket is stationary, and the orientation is in it's initial state.

$$\text{reference} := \boldsymbol{\psi}_0$$

The translation that relates the magnetic field to the orientation of the rocket is

$$\hat{\boldsymbol{\psi}} = \mathbf{q} \boldsymbol{\psi}_0 \bar{\mathbf{q}}$$

In other terms, the system thinks the output of the magnetometer will be $\hat{\boldsymbol{\psi}}$.

Anytime we want to measure the orientation of the rocket, we take a reading of the magnetic field $\boldsymbol{\psi}$ and compare it to the prediction.

$$\hat{\boldsymbol{\psi}} = \mathbf{q} \boldsymbol{\psi}_0 \bar{\mathbf{q}}$$

$$\boldsymbol{\varepsilon} = \boldsymbol{\psi} - \hat{\boldsymbol{\psi}}$$

The error $\boldsymbol{\varepsilon}$ is the difference between the measured orientation and the predicted orientation. The error is then used to correct the orientation of the rocket. We will see how this is done in the Kalman filter section.

Similar to the jacobian for the velocity dynamics, we can linearize the orientation measurement model to get the jacobian matrix

$$\mathbf{H}_q = \begin{pmatrix} q_1\psi_x + q_3\psi_z - q_4\psi_y & q_2\psi_x + q_3\psi_y + q_4\psi_z & q_1\psi_z - q_3\psi_x + q_2\psi_y & q_4\psi_x - q_1\psi_y + q_2\psi_z \\ q_1\psi_y + q_4\psi_x - q_2\psi_z & q_3\psi_x - q_2\psi_y - q_1\psi_z & q_3\psi_y + q_2\psi_x + q_4\psi_z & q_3\psi_z - q_4\psi_y + q_1\psi_x \\ q_1\psi_z + q_2\psi_y - q_3\psi_x & q_4\psi_x - q_2\psi_z + q_1\psi_y & q_4\psi_y - q_3\psi_z - q_1\psi_x & q_4\psi_z + q_2\psi_x + q_3\psi_y \end{pmatrix}$$

Altitude measurement

The altitude can be measured by a barometer. The barometer measures the pressure of the air. The pressure of the air decreases with altitude.

$$h = h_0 + l_0 \left(\frac{p_0}{p} \right)^{\frac{m_0}{\tau_0} l_0}$$

As we lack a GPS, we cannot measure the velocity of the rocket directly. The only way to update the velocity is via the dynamics of the system. This is a common problem in navigation systems and is called the observability problem. The velocity is not observable from the sensors, and we must rely on the dynamics of the system

$$\frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \mathbf{H} = \begin{pmatrix} \mathbf{H}_q & \mathbf{0} \\ \mathbf{0} & h \end{pmatrix}$$

State estimation with Kalman filter

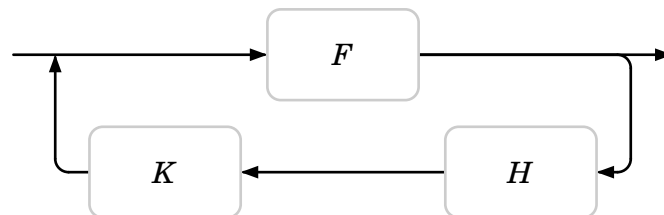
With the state-space representation of the system and the linearized dynamics, we can use a Kalman filter to estimate the state of the rocket. For pure linear systems, the Kalman filter is optimal and can be used to estimate the state of the system given noisy measurements. However, the system is not purely linear, so we must use an extended Kalman filter which is not optimal but does a good job of estimating the state of the system nonetheless. The extended Kalman filter is identical to the Kalman filter except for the linearization step. The Kalman filter functions kinda like a feedback controller. It takes the measurements and the estimated state and corrects the estimated state based on the error between the measurements. Recall the system dynamics (1)

$$\begin{aligned} \mathbf{x}_{t+1} &= \mathbf{F}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t + \mathbf{w}_d \\ \hat{\mathbf{y}}_t &= \mathbf{H}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t + \mathbf{w}_n \end{aligned}$$

We can add the feedback loop to the system dynamics to get the Kalman filter

$$\hat{\mathbf{x}}_{t+1} = \mathbf{F}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t + \mathbf{K}(\mathbf{y} - \hat{\mathbf{y}})$$

where $\mathbf{y}$ is the actual measurement and $\hat{\mathbf{y}}$ is the predicted measurement from our measurement model. Below is a very barebones illustration of the Kalman filter



The choice of the gain K is not arbitrary, it is chosen to minimize the cost function

$$J = E[\mathbf{x} - \hat{\mathbf{x}}]^T Q E[\mathbf{x} - \hat{\mathbf{x}}] + E[\mathbf{y} - \hat{\mathbf{y}}]^T R E[\mathbf{y} - \hat{\mathbf{y}}]$$

Recall the state-space representation of the system equation (1) where we have system disturbances $\mathbf{w}_d$ and measurement noise $\mathbf{w}_n$. Now a very important assumption that the Kalman filter bases its optimality on is that the noise is Gaussian.

which is the expected value of the error between the true state and the estimated state. This is a quadratic cost function that is minimized by the Kalman filter. By solving the Riccati equation, we can find the optimal Kalman gain K

Hardware

The PCB is designed in KiCad and is a 4-layer board. The board is powered by a 3.3V LDO regulator. In general, the PCB design is centered around the ARM Cortex M4 microcontroller and the sensors. The sensors are connected to the microcontroller via a shared SPI bus, so the microcontroller has to select the sensor it wants to communicate with by setting the chip select pin of the sensor. The only exception is the magnetometer which is connected via I2C directly to the ISM330DHCX which have an I2C bus and DSP functionality. Most ICs have a decoupling capacitors as recommended by the manufacturer.

ARM Cortex M4

The ARM Cortex M4 is a 32-bit microcontroller that is used to run the navigation system. The processing power of the microcontroller is sufficient to run the navigation algorithms and communicate with the sensors. The peripherals used by the microcontroller are the SPI, USART, Timer, and SD card. It also provides an interrupt vector table that we use for the state machine. The sensor drivers are only a matter of reading and writing to the registers of the sensors with SPI

Firmware

The codebase uses the harmony framework to interface with the peripherals of the microcontroller. Harmony is a code generator that generates code based on the configuration of the peripherals. It provides a high-level API to interface with the peripherals of the microcontroller.

The main loop of the firmware is a state machine that runs the navigation algorithms. The state machine is implemented as a switch-case statement that switches between the different states of the system. Some states are triggered by interrupts from the sensors, while others are triggered by a timer. The state machine is responsible for reading the sensor data, running the navigation algorithms, and sending the data to the ground station.

API

The API of the firmware is quite limited. The firmware is designed to be a standalone system that runs the navigation algorithms and communicates with the sensors. The only way to interact with the firmware is through the SERCOM USART interface. The firmware sends the data to the ground station in a binary format. The data is sent in packets that contain the sensor data and the estimated state of the rocket.

Testing

The most useful tests are the SERCOM interface in debug mode. By connecting the system to a computer we can print useful information about the state of the system. The tests are run in the lab and in the field. The project also comes with visualizations of the data that can be used to analyze the performance of the system, and unit tests that can be run independently of the microcontroller. sensor specific tests are also run to ensure that the sensors are working as expected.

Integration

This section is about how the system is integrated into the rocket. How the system is powered and how it communicates with the ground station and other systems. We initially used FRAM but switched to usart for simplicity and higher data rates as the FRAM was slow. It is connected directly to the recovery system and the RF module. The physical position of the system is together with the rest of the electronics, ideally in the center of mass of the rocket. Shielding is also preferred to limit sensor noise and interference.